# Measuring and Understanding Throughput of Network Topologies

Sangeetha Abdu Jyothi*, Ankit Singla†, P. Brighten Godfrey*, Alexandra Kolla*
*University of Illinois at Urbana–Champaign †ETH Zurich

***Abstract*—High throughput is of particular interest in data center and HPC networks. Although myriad network topologies have been proposed, a broad head-to-head comparison across topologies and across traffic patterns is absent, and the right way to compare worst-case throughput performance is a subtle problem.**

**In this paper, we develop a framework to benchmark the throughput of network topologies, using a two-pronged approach. First, we study performance on a variety of synthetic and experimentally-measured traffic matrices (TMs). Second, we show how to measure worst-case throughput by generating a near-worst-case TM for any given topology. We apply the framework to study the performance of these TMs in a wide range of network topologies, revealing insights into the performance of topologies with scaling, robustness of performance across TMs, and the effect of scattered workload placement. Our evaluation code is freely available.**

## I. Introduction

Throughput is a fundamental property of communication networks: at what rate can data be carried across the network between desired end-points? Particularly for data centers and high performance computing, an increase in throughput demand among compute elements has reinvigorated research on network topology, and a large number of network topologies have been proposed in the past few years to achieve high capacity at low cost [3], [8], [9], [11], [14]–[16], [30], [38], [39], [43], [44].

However, there is little order to this large and ever-growing set of network topologies. We lack a broad comparison of topologies, and there is no open, public framework available for testing and comparing topology designs. The absence of a well-specified benchmark complicates research on network design, making it difficult to evaluate a new design against the numerous past proposals, and difficult for industry to know which threads of research are most promising to adopt.

Our goal is to build a framework for accurate and consistent measurement of the throughput of network topologies, and use this framework to benchmark proposed data center and HPC topologies.

To accomplish this, we need metrics for comparison of throughput, and this turns out to be a subtle problem. Throughput can be measured by testing particular workloads, or traffic matrices (TMs), but the immediate question is what TMs to test. One approach is to test a variety of common TMs, which can provide insight into the effect of topological structure for particular use cases reflected by those specific TMs. However, we argue it is useful to go beyond this. In HPC and data center networks, TMs may vary widely depending on the use-case and across time as applications spin up and down or migrate [14], [18]–[21]. Although some applications map well onto certain topologies and known worst-case traffic patterns [33], [42] can be avoided in such cases, a mix of multiple applications could still produce an unintended difficult TM. In fact, [19] observes that network contention among applications sharing an HPC system will worsen in the near future (even after accounting for the expected increase in network speeds) as HPC applications grow in size and variety. Hence, it is useful to understand the *worst-case* performance of a topology, for any TM. However, currently, there does not exist a systematic way to evaluate the worst-case throughput achievable in a network and to identify the traffic pattern responsible for it.

Our key contributions, then, are to (1) develop a heuristic to measure worst-case throughput, and (2) provide an expansive benchmarking of a variety of topologies using a variety of TMs – TMs generated from real-world measurements, synthetic TMs, and finally our new near-worst-case TMs. We discuss each of these in more detail.

(1a) **We evaluate whether cut-based metrics**, such as bisection bandwidth and sparsest cut, solve the problem of estimating worst-case throughput. A number of studies (e.g. [2], [8], [40], [41], [45]) employ cut metrics. It has been noted [46] that bisection bandwidth does not always predict average-case throughput (in a limited setting; §V). But do cuts measure *worst-case* throughput? We show that it does not, by proving the existence of two families of networks, $A$ and $B$, where $A$ has a higher cut-metric even though $B$ supports *asymptotically* higher worst-case throughput. Further, we show that the mismatch between cuts and worst-case throughput exists even for highly-structured networks of small size – a 5-ary 3-stage butterfly with only 25 nodes – where the sparsest-cut found through brute force computation is strictly greater than the worst-case throughput.

(1b) Since cut metrics don't achieve our goal, **we develop a heuristic to measure worst-case throughput**. We propose an efficient algorithm to generate a near-worst-case TM for any given topology. We show empirically that these near-worst-case TMs approach a theoretical lower bound of throughput in practice. Note that Kodialam et. al. [25] previously developed a TM for a somewhat different reason[1] which could be re-

[1]Upper-bounding throughput performance of a routing scheme using an LP formulation. In addition, [25] did not use their TM to benchmark topologies.

SC16; Salt Lake City, Utah, USA; November 2016
978-1-4673-8815-3/16/$31.00 ©2016 IEEE



Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.

purposed as a near-worst-case TM. Compared with [25], our methodology finds TMs that are just as close to the worst case, can be computed approximately $6\times$ faster, and scales to networks $8\times$ larger with the same memory limit.

(2) We perform a **head-to-head benchmark of a wide range of topologies** across a variety of workloads. Specifically, we evaluate a set of 10 topologies proposed for data centers and high performance computing, with our near-worst-case TMs, a selection of synthetic TMs, and real TMs measured from two operational Facebook data centers. Key findings of this study include:

- **Which topologies perform well?** We find the answer depends on scale. At small scale, several topologies, particularly DCell, provide excellent performance across nearly all TMs. However, at $\geq$ 1000s of servers, proposals based on expander graphs (Jellyfish, Long Hop, Slim Fly) provide the best performance, with the advantage increasing with scale (an observation confirmed by a recent data center proposal [43]).
  These results add new information compared with past work. We find the three expanders have nearly identical performance for uniform traffic (e.g. all-to-all); this differs from conclusions of [4], [41] which analyzed bisection bandwidth rather than throughput. We find that expanders significantly outperform fat trees; this differs from the results of [47] which concluded that fat trees perform as well or better. We replicate the result of [47] and show the discrepancy results from an inaccurate calculation of throughput and unequal topology sizes.
- **What topologies have robustly high performance even with worst-case traffic?** The effect of near-worst-case traffic differs. We find that certain topologies (Dragonfly, BCube) are less robust, dropping more substantially in performance. Jellyfish and Long Hop are most robust.
  While fat trees are otherwise generally robust, we show that under a TM with a small fraction of large flows in the network, fat trees perform poorly. We find this is due to lack of path diversity near the endpoints.
- **How do real-world workloads perform?** We experiment with real world TMs from two different clusters based on data from Facebook [34]. Surprisingly, in one very nonuniform TM we find that randomizing rack-level workload placement consistently *improves* performance for all networks we tested other than fat trees and the expanders. The fact that expanders and fat trees perform well even without this workload rearrangement is consistent with our finding that they are robust to worst-case performance. But also, for the other topologies, it indicates an intriguing opportunity for improving performance through better workload placement.

To the best of our knowledge, this work is the most expansive comparison of network topology throughput to date, and the only one to use accurate and consistent methods. Our results provide insights that are useful in topology selection and workload placement. But just as importantly, our evaluation framework and the set of topologies tested are freely available [1]. We hope our tools will facilitate future work on rigorously designing and evaluating networks, and replication of research results.

## II. Metrics for throughput

In this section, we define throughput precisely. We then study how to evaluate worst-case throughput: we consider cut-metrics, before presenting a heuristic algorithm that produces a near-worst-case traffic matrix.

### A. Throughput defined

Our focus in this paper is on network throughput. Furthermore, our focus is on *network topology*, not on higher-level design like routing and congestion control.[2] Therefore, the metric of interest is end-to-end throughput supported by a network in a fluid-flow model with optimal routing. We next define this more precisely.

A **network** is a graph $G = (V, E)$ with capacities $c(u, v)$ for every edge $(u, v) \in E_G$. Among the nodes $V$ are **servers**, which send and receive traffic flows, connected through non-terminal nodes called **switches**. Each server is connected to one switch, and each switch is connected to zero or more servers, and other switches. Unless otherwise specified, for switch-to-switch edges $(u, v)$, we set $c(u, v) = 1$, while server-to-switch links have infinite capacity. This allows us to stress-test the network topology itself, rather than the servers.

A **traffic matrix (TM)** $T$ defines the traffic demand: for any two servers $v$ and $w$, $T(v, w)$ is an amount of requested flow from $v$ to $w$. We assume without loss of generality that the traffic matrix is normalized so that it conforms to the "hose model": each server sends at most 1 unit of traffic and receives at most 1 unit of traffic ($\forall v$, $\sum_w T(v, w) \leq 1$ and $\sum_w T(w, v) \leq 1$).

The **throughput** of a network $G$ with TM $T$ is the maximum value $t$ for which $T \cdot t$ is feasible in $G$. That is, we seek the maximum $t$ for which there exists a feasible multicommodity flow that routes flow $T(v, w) \cdot t$ through the network from each $v$ to each $w$, subject to the link capacity and the flow conservation constraints. This can be formulated in a standard way as a linear program (omitted for brevity) and is thus computable in polynomial time. If the nonzero traffic demands $T(v, w)$ are equal, this is equivalent to the *maximum concurrent flow* [35] problem: maximizing the minimum throughput of any requested end-to-end flow.

Note that we could alternately maximize the *total* throughput of all flows. We avoid this because it would allow the network to pick and choose among the TM's flows, giving high bandwidth only to the "easy" ones (e.g., short flows that do not cross a bottleneck). The formulation above ensures that *all* flows in the given TM can be satisfied at the desired rates specified by the TM, all scaled by a constant factor.

We now have a precise definition of throughput, but it depends on the choice of TM. How can we evaluate a

[2]There are many performance measures for networks that are not considered here, and higher-layer systems obviously affect throughput experienced by applications. But our goal is a comprehensive understanding of the fundamental goal of high-throughput network topology. At this stage, systems-level issues would be a distraction.



Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.

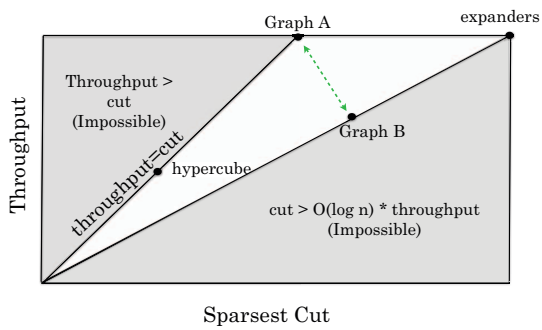



Fig. 1. Sparsest cut vs. Throughput

*topology* independent of assumptions about the traffic, if we are interested in worst-case traffic?

### B. Cuts: a weak substitute for worst-case throughput

Cuts are generally used as proxies to estimate throughput. Since any cut in the graph upper-bounds the flow across the cut, if we find the minimum cut, we can bound the worst-case performance. Two commonly used cut metrics are:

*(a) Bisection bandwidth:* It is a widely used to provide an evaluation of a topology's performance independent of a specific TM. It is the capacity of the worst-case cut that divides the network into two equal halves ( [31], p. 974).

*(b) Sparsest cut:* The sparsity of a cut is the ratio of its capacity to the net weight of flows that traverse the cut, where the flows depend on a particular TM. Sparsest cut refers to the minimum sparsity in the network. The special case of *uniform sparsest cut* assumes the all-to-all TM.

Cuts provide an upper-bound on worst-case network performance, are simple to state, and can sometimes be calculated with a formula. However, they have several limitations.

**(1) Sparsest cut and bisection bandwidth are not actually TM-independent**, contrary to the original goal of evaluating a topology independent of traffic. Bisection bandwidth and the uniform sparsest cut correspond to the worst cuts for the complete (all-to-all) TM, so they have a hidden implicit assumption of this particular TM.

**(2) Even for a specific TM, computing cuts is NP-hard**, and it is believed that there is no efficient constant factor approximation algorithm [6], [13]. In contrast, throughput is computable in polynomial time for any specified TM.

**(3) Cuts are only a *loose* upper-bound for worst-case throughput.** This may be counter-intuitive if our intuition is guided by the well-known max-flow min-cut theorem which states that in a network with a single flow, the maximum achievable flow is equal to the minimum capacity over all cuts separating the source and the destination [10], [12]. However, this no longer holds when there are more than two flows in the network, i.e., *multi-commodity flow*: the maximum flow (throughput) can be an $O(\log n)$ factor lower than the sparsest cut [28]. Hence, cuts do not directly capture the maximum flow.

Figure 1 depicts this relationship between cuts and throughput. Here we focus on sparsest cut.[3] The flow (throughput) in the network cannot exceed the upper bound imposed by the worst-case cut. On the other hand, the cut cannot be more than a factor $O(\log n)$ greater than the flow [28]. Thus, any graph and an associated TM can be represented by a unique point in the region bounded by these limits.

While this distinction is well-established [28], we strengthen the point by showing that *it can lead to incorrect decisions when evaluating networks*. Specifically, we will exhibit a pair of graphs $A$ and $B$ such that, as shown in Figure 1, $A$ has higher throughput but $B$ has higher sparsest cut. If sparsest cut is the metric used to choose a network, graph $B$ will be wrongly assessed to have better performance than graph $A$, while in fact it has a factor $\Omega(\sqrt{\log n})$ worse performance!

**Graph A** is a clustered random graph adapted from previous work [37] with $n$ nodes and degree $2d$. $A$ is composed of two equal-sized clusters with $n/2$ nodes each. Each node in a cluster is connected by degree $\alpha$ to nodes inside its cluster, and degree $\beta$ to nodes in the other cluster, such that $\alpha+\beta = 2d$. $A$ is sampled uniformly at random from the space of all graphs satisfying these constraints. We can pick $\alpha$ and $\beta$ such that $\beta = \Theta(\frac{\alpha}{\log n})$. Then, as per [37] (Lemma 3), the throughput of $A$ (denoted $T_A$) and its sparsest cut (denoted $\Phi_A$) are both $\Theta(\frac{1}{n \log n})$.

Let graph $G$ be any $2d$-regular expander on $N = \frac{n}{dp}$ nodes, where $d$ is a constant and $p$ is a parameter we shall adjust later. **Graph B** is constructed by replacing each edge of $G$ with a path of length $p$. It is easy to see that $B$ has $n$ nodes. We prove in the tech report [22], the following theorem.

**Theorem 1.** $T_B = O(\frac{1}{np\log n})$ *and* $\Phi_B = \Omega(\frac{1}{np})$.

In the above, setting $p = 1$ corresponds to the 'expanders' point in Figure 1: both $A$ and $B$ have the same throughput (within constant factors), but the $B$'s cut is larger by $O(\log n)$. Increasing $p$ creates an asymptotic separation in both the cut and the throughput such that $\Phi_A < \Phi_B$, while $T_A > T_B$.

**Intuition.** The reason that throughput may be smaller than sparsest cut is that in addition to being limited by bottlenecks, the network is limited by the total volume of "work" it has to accomplish within its total link capacity. That is, if the TM has equal-weight flows,

$$Throughput\ per\ flow \leq \frac{Total\ link\ capacity}{\#\ of\ flows\ \cdot\ Avg\ path\ length}$$

where the total capacity is $\sum_{(i,j)\in E} c(i,j)$ and the average path length is computed over the flows in the TM. This "volumetric" upper bound may be tighter than a cut-based bound.

Based on the above result and experimental confirmation in §III, we conclude that cuts are not a good choice to evaluate throughput. Cuts still capture important topological properties such as the difficulty of partitioning a network, and physical layout (packaging and cabling) complexity. However, these properties are outside the scope of our study of throughput.

[3]We pick one for simplicity, and sparsest cut has an advantage in robustness. Bisection bandwidth is forced to cut the network in equal halves, so it can miss more constrained bottlenecks that cut a smaller fraction of the network.



Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.

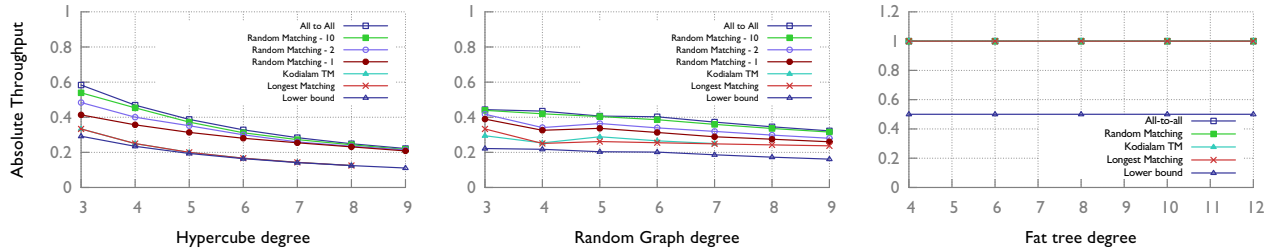



Fig. 2. Throughput resulting from several different traffic matrices in three topologies

### C. Towards a worst-case throughput metric

Having exhausted cut-based metrics, we return to the original metric of throughput defined in §II-A. We can evaluate network topologies directly in terms of throughput (via LP optimization software) for specific TMs. The key, of course, is how to choose the TM. Our evaluation can include a variety of synthetic and real-world TMs, but as discussed in the introduction, we also want to evaluate topologies' robustness to *unexpected* TMs.

If we can find a worst-case TM, this would fulfill our goal. However, computing a worst-case TM is an unsolved, computationally non-trivial problem [7].[4] Here, we offer an efficient heuristic to find a *near-worst-case TM* which can be used to benchmark topologies.

We begin with the complete or **all-to-all TM** $T_{A2A}$ which for all $v, w$ has $T_{A2A}(v, w) = \frac{1}{n}$. We observe that $T_{A2A}$ is within $2\times$ of the worst case TM. This fact is simple and known to some researchers, but at the same time, we have not seen it in the literature, so we give the statement here and proof in a tech report [22].

**Theorem 2.** *Let $G$ be any graph. If $T_{A2A}$ is feasible in $G$ with throughput $t$, then any hose model traffic matrix is feasible in $G$ with throughput $\geq t/2$.*

Can we get closer to the worst case TM? In our experience, TMs with a smaller number of "elephant" flows are more difficult to route than TMs with a large number of small flows, like $T_{A2A}$. This suggests a **random matching TM** in which we have only one outgoing flow and one incoming flow per server, chosen uniform-randomly among servers.

Can we get *even closer* to the worst-case TM? Intuitively, the all-to-all and random matching TMs will tend to find sparse cuts, but only have average-length paths. Drawing on the intuition that throughput decreases with average flow path length, we seek to produce traffic matrices that force the use of long paths. To do this, given a network $G$, we compute all-pairs shortest paths and create a complete bipartite graph $H$, whose nodes represent all sources and destinations in $G$, and for which the weight of edge $v \to w$ is the length of the shortest $v \to w$ path in $G$. We then find the maximum weight matching in $H$. The resulting matching corresponds to the pairing of servers which maximizes average flow path length, assuming flow is routed on shortest paths between each pair. We call this a **longest matching TM**, and it will serve as our heuristic for a near-worst-case traffic.

Kodialam et al. [25] proposed another heuristic to find a near-worst-case TM: maximizing the average path length of a flow. This **Kodialam TM** is similar to the longest matching but may have many flows attached to each source and destination. This TM was used in [25] to evaluate oblivious routing algorithms, but there was no evaluation of how close it is to the worst case, so our evaluation here is new.

Figure 2 shows the resulting throughput of these TMs in three topologies: hypercubes, random regular graphs, and fat trees [3]. In all cases, A2A traffic has the highest throughput; throughput decreases or does not change as we move to a random matching TM with 10 servers per switch, and progressively decreases as the number of servers per switch is decreased to 1 under random matching, and finally to the Kodialam TM and the longest matching TM. We also plot the lower bound given by Theorem 2: $T_{A2A}/2$. Comparison across topologies is not relevant here since the topologies are not built with the same "equipment" (node degree, number of servers, etc.)

We chose these three topologies to illustrate cases when our approach is most helpful, somewhat helpful, and least helpful at finding near-worst-case TMs. In the **hypercube**, the longest matching TM is extremely close to the worst-case performance. To see why, note that the longest paths have length $d$ in a $d$-dimensional hypercube, twice as long as the mean path length; and the hypercube has $n \cdot d$ unidirectional links. The total flow in the network will thus be $(\#\, flows \cdot average\, flow\, path\, length) = n \cdot d$. Thus, all links will be perfectly utilized if the flow can be routed, which empirically it is. In the **random graph**, there is less variation in end-to-end path length, but across our experiments the longest matching is always within $1.5\times$ of the provable lower bound (and may be even closer to the true lower bound, since Theorem 2 may not be tight). In the **fat tree**, which is here built as a three-level nonblocking topology, there is essentially no variation in path length since asymptotically nearly all paths go three hops up to the core switches and three hops down to the destination. Here, our TMs are no worse than all-to-all, and the simple $T_{A2A}/2$ lower bound is off by a factor of 2 from the true worst case (which is throughput of 1 as this is a nonblocking topology).

[4]Our problem corresponds to the separation problem of the minimum-cost robust network design in [7]. This problem is shown to be hard for the single-source hose model. However, the complexity is unknown for the hose model with multiple sources which is the scenario we consider.



Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.

The longest matching and Kodialam TMs are identical in hypercubes and fat trees. On random graphs, they yield slightly different TMs, with longest matching yielding marginally better results at larger sizes. In addition, longest matching has a significant practical advantage: it produces far fewer end-to-end flows than the Kodialam TM. Since the memory requirements of computing throughput of a given TM (via the multicommodity flow LP) depends on the number of flows, longest matching requires less memory and compute time. For example, in random graphs on a 32 GB machine using the Gurobi optimization package, the Kodialam TM can be computed up to 128 nodes while the longest matching scales to 1,024, while being computed roughly $6\times$ faster. Hence, we choose longest matching as our representative near-worst-case traffic matrix.

### D. Summary and implications

Directly evaluating throughput with particular TMs using LP optimization is both more accurate and more tractable than cut-based metrics. In choosing a TM to evaluate, both "average case" and near-worst-case TMs are reasonable choices. Our evaluation will employ multiple synthetic TMs and measurements from real-world applications. For near-worst-case traffic, we developed a practical heuristic, the longest matching TM, that often succeeds in substantially worsening the TM compared with A2A.

Note that measuring throughput directly (rather than via cuts) is not in itself a novel idea: numerous papers have evaluated particular topologies on particular TMs. Our contribution is to provide a rigorous analysis of why cuts do not always predict throughput; a way to generate a near-worst-case TM for any given topology; and an experimental evaluation benchmarking a large number of proposed topologies on a variety of TMs.

## III. Experimental Methods

In this section, we present our experimental methodology and detailed analysis of our framework. We attempt to answer the following questions: Are cut-metrics indeed worse predictors of performance? When measuring throughput directly, how close do we come to worst-case traffic?

### A. Methodology

Before delving into the experiments, we explain the methods used for computing throughput. We also elaborate on the traffic workloads and topologies used in the experiments.

*1) Computing throughput:* Throughput is computed as a solution to a linear program whose objective is to maximize the minimum flow across all flow demands, as explained in §II. We use the Gurobi [17] linear program solver. Throughput depends on the traffic workload provided to the network.

*2) Traffic workload:* We evaluate two main categories of workloads: (a) real-world measured TMs from Facebook clusters and (b) synthetic TMs, which can be uniform weight or non-uniform weight. Synthetic workloads belong to three main families: all-to-all, random matching and longest matching (near-worst-case). In addition, we need to specify where the traffic endpoints (sources and destinations, i.e., servers) are attached. In structured networks with restrictions on server-locations (fat-tree, BCube, DCell), servers are added at the locations prescribed by the models. For example, in fat-trees, servers are attached only to the lowest layer. For all other networks, we add servers to each switch. Note that our traffic matrices effectively encode switch-to-switch traffic, so the particular *number* of servers doesn't matter.

*3) Topologies:* Our evaluation uses 10 families of computer networks. Topology families evaluated are: BCube [15], DCell [16], Dragonfly [24], Fat Tree [29], Flattened butterfly [23], Hypercubes [5], HyperX [2], Jellyfish [38], Long Hop [41] and Slim Fly [4]. For evaluating the cut-based metrics in a wider variety of environments, we consider 66 non-computer networks – food webs, social networks, and more [1].

### B. Do cuts predict worst-case throughput?

In this section, we experimentally evaluate whether cut-based metrics predict throughput. We generate multiple networks from each of our topology families (with varying parameters such as size and node degree), compute throughput with the longest matching TM, and find sparse cuts using heuristics with the same longest matching TM. We show that:

- In several networks, bisection bandwidth cannot predict throughput accurately. For a majority of large networks, our best estimate of sparsest-cut differs from the computed worst-case throughput by a large factor.
- Even in a well-structured network of small size (where brute force is feasible), sparsest-cut can be higher than worst-case throughput.

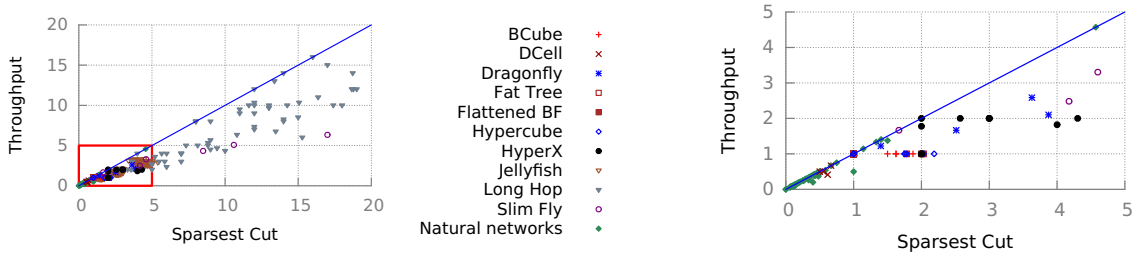
Since brute-force computation of cuts is possible only on small networks (up to 20 nodes), we evaluate bisection bandwidth and sparsest cut on networks of feasible size (115 networks total – 100 Jellyfish networks and 15 networks from 7 families of topologies). Of the 8 topology families tested, we found that bisection bandwidth accurately predicted throughput in only 5 of the families while sparsest cut gives the correct value in 7. The average error (difference between cut and throughput) is $7.6\%$ for bisection bandwidth and $6.2\%$ for sparsest cut (in networks where they differ from throughput). Maximum error observed was $56.3\%$ for bisection bandwidth and $6.2\%$ for sparsest cut.

Although sparsest cut does a better job at estimating throughput at smaller sizes, we have found that in a 5-ary 3-stage flattened butterfly with only 25 switches and 125 servers, the throughput is less than the sparsest cut (and the bisection bandwidth). Specifically, the absolute throughput in the network is $0.565$ whereas the sparsest-cut is $0.6$. This shows that even in small networks, throughput can be different from the worst-case cut. While the differences are not large in the small networks where brute force computation is feasible, note that since cuts and flows are separated by an $\Theta(\log n)$ factor, we should expect them to grow.

Sparsest cut being the more accurate cut metric, we extend the sparsest cut estimation to larger networks. Here we have



Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.




(a) Throughput vs. cut for all graphs

(b) Throughput vs. cut for selected graphs (zoomed version of (a))

Fig. 3. Throughput vs. cut. (Comparison is valid for individual networks, not *across* networks, since they have different numbers of nodes and degree distributions.)

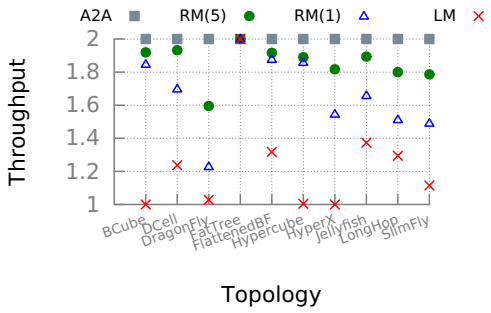



Fig. 4. Comparison of throughput under different traffic matrices normalized with respect to theoretical lower bound

to use heuristics to compute sparsest cut, but we compute all of an extensive set of heuristics (limited brute-force, selective brute-force involving one or two nodes in a subset, eigenvector-based optimization, etc.) and use the term **sparse cut** to refer to the sparsest cut that was found by any of the heuristics. Sparse cuts differ from throughput substantially, with up to a $3\times$ discrepancy as shown in Figure 3. In only a small number of cases, the cut equals throughput. The difference between cut and throughput is pronounced. For example, Figure 3(b) shows that although HyperX networks of different sizes have approximately same flow value (y axis), they differ widely in sparsest cut (x axis). This shows that estimation of worst-case throughput performance of networks using cut-based metrics can lead to erroneous results.

### C. Does longest matching approach the true worst case?

We compare representative samples from each family of topology under four types of TM: all to all (A2A), random matching with 5 servers per switch, random matching with 1 server per switch, and longest matching. Figure 4 shows the throughput values normalized so that the theoretical lower bound on throughput is 1, and therefore A2A's throughput is 2. For all networks, $T_{A2A} \geq T_{RM(5)} \geq T_{RM(1)} \geq T_{LM} \geq 1$, i.e., all-to-all is consistently the easiest TM in this set, followed by random matchings, longest matching, and the theoretical lower bound. This confirms our intuition discussed in §II-C. (As in Figure 3, throughput comparisons are valid across TMs for a particular network, not across networks since the number of links and servers varies across networks.)

Our longest matching TM is successful in matching the lower bound for BCube, Hypercube, HyperX, and (nearly) Dragonfly. In all other families except fat trees, the traffic under longest matching is significantly closer to the lower bound than with the other TMs. In fat trees, throughput under A2A and longest matching are equal. This is not a shortcoming of the metric, rather its the lower bound which is loose here: in fat trees, it can be easily verified that the flow out of each top-of-rack switch is the same under all symmetric TMs (i.e., with equal-weight flows and the same number of flows into and out of each top-of-rack switch).

In short, these results show that (1) the heuristic for near-worst-case traffic, the longest matching TM, is a significantly more difficult TM than A2A and RM and often approaches the lower bound; and (2) throughput measurement using longest matching is a more accurate estimate of worst-case throughput performance than cut-based approximations, in addition to being substantially easier to compute.

## IV. Topology Evaluation

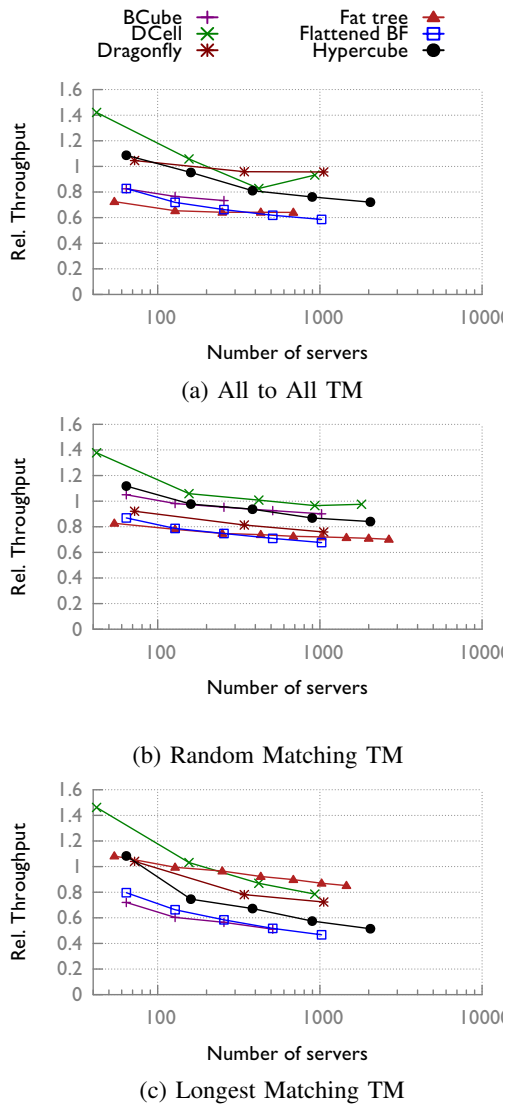
In this section, we present the results of our topology evaluation with synthetic and real-world workloads, and our near-worst-case TM.

But first, there is one more piece of the puzzle to allow comparison of networks. Networks may be built with different equipment – with a wide variation in number of switches and links. The raw throughput value does not account for this difference in hardware resources, and most proposed topologies can only be built with particular discrete numbers of servers, switches, and links, which inconveniently do not match.

Fortunately, uniform-random graphs as in [38] can be constructed for any size and degree distribution. Hence, random graphs serve as a convenient benchmark for easy comparison of network topologies. Our high-level approach to evaluating a network is to: (i) compute the network's throughput; (ii) build a random graph with precisely the same equipment, i.e., the same number of nodes each with the same number of links



Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.

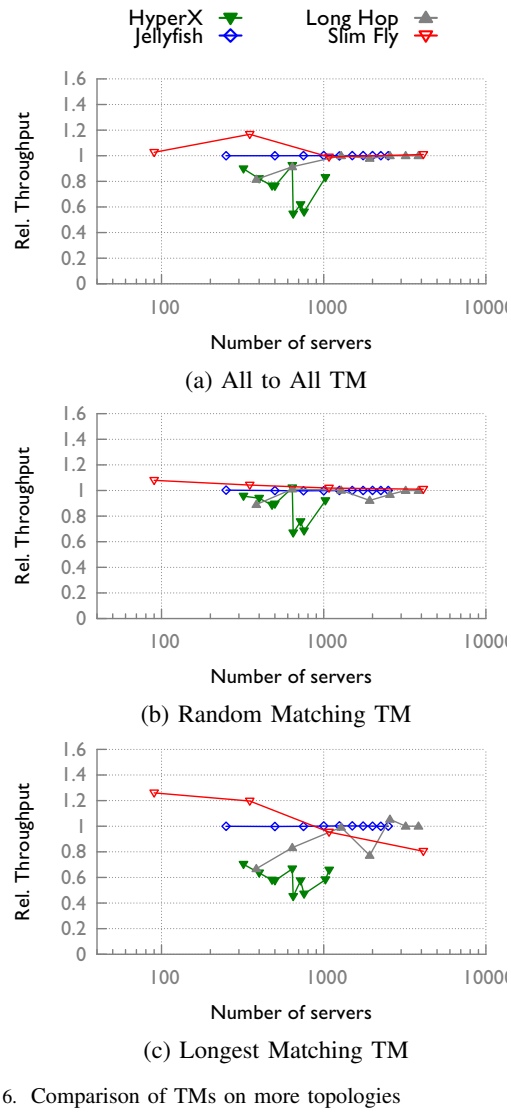



(a) All to All TM

(b) Random Matching TM

(c) Longest Matching TM

Fig. 5. Comparison of TMs on topologies




(a) All to All TM

(b) Random Matching TM

(c) Longest Matching TM

Fig. 6. Comparison of TMs on more topologies

as the corresponding node in the original graph, (iii) compute the throughput of this same-equipment random graph under the same TM; (iv) normalize the network's throughput with the random graph's throughput for comparison against other networks. This normalized value is referred to as **relative throughput**. Unless otherwise stated, each data-point is an average across 10 iterations, and all error bars are 95% two-sided confidence intervals. Minimal variations lead to narrow error bars in networks of size greater than 100.

### *A. Synthetic Workloads*

We evaluate the performance of 10 types of computer networks under two categories of synthetic workloads: (a) uniform weight and (b) non-uniform weight.

#### *1) Uniform-Weight Synthetic Workloads*

We evaluate three traffic matrices with equal weight across flows: all to all, random matching with one server, and longest matching. Figures 5 and 6 show the results, divided among two figures for visual clarity. Most topologies are in Figure 5, while Figure 6 shows Jellyfish, Slim Fly, Long Hop, and HyperX.

**Overall,** performance varies substantially, by around $1.6\times$ with A2A traffic and more than $2\times$ with longest matching. Furthermore, for the networks of Figure 5, which topology "wins" depends on the TM. For example, Dragonfly has high relative throughput under A2A but under the longest matching TM, fat trees achieve the highest throughput at the largest scale. However, in all TMs, Jellyfish achieves highest performance (relative throughput $= 1$ by definition), with longest matching at the largest scale (1000+ servers). In comparison with random graphs, performance degrades for most networks with increasing size. The relative performance of the largest network tested in each family in Figure 5 is given in Table IV-A. The pattern of performance of the networks shown in Figure 6 is different, so we discuss these in detail



Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.

next.

| Topology family | All-To-All | Random Matching | Longest matching |
|---|---|---|---|
| BCube (2-ary) | 73% | 90% | 51% |
| DCell (5-ary) | 93% | 97% | 79% |
| Dragonfly | 95% | 76% | 72% |
| Fat tree | 65% | 73% | 89% |
| Flattened BF (2-ary) | 59% | 71% | 47% |
| Hypercube | 72% | 84% | 51% |

TABLE I
RELATIVE THROUGHPUT AT THE LARGEST SIZE TESTED UNDER DIFFERENT TMS

**HyperX** [2] has irregular performance across scale. The variations arise from the choice of underlying topology: Given a switch radix, number of servers and desired bisection bandwidth, HyperX attempts to find the least cost topology constructed from the building blocks – hypercube and flattened butterfly. Even a slight variation in one of the parameters can lead to a significant difference in HyperX construction and hence throughput.

We plot HyperX results for networks with bisection bandwidth of $0.4$, i.e., in the worst case the network can support flow from $40\%$ of the hosts without congestion. In the worst case, the HyperX network that can support $648$ hosts achieves a relative throughput of $55\%$ under all-to-all traffic. For the same network, relative performance improves to $67\%$ under random matching but further degrades to $45\%$ under longest matching. We investigate the performance of HyperX networks with different bisection bandwidths under the longest matching TM in Figure 7 and observe that the performance varies widely with network size under all values of bisection. More importantly, this further illustrates that high bisection does not guarantee high performance.

**Jellyfish** [38] is our benchmark, and its (average) relative performance is $100\%$ under all workloads by definition. Performance depends on randomness in the construction, but at larger sizes, performance differences between Jellyfish instances are minimal ($\approx 1\%$).

**Long Hop networks** [41] In Figure 8, we show that the relative throughput of Long Hop networks approaches $1$ at large sizes under the longest matching TM. Similar trends are observed under all-to-all and random matching workloads. The paper [41] claimed high performance (in terms of bisection bandwidth) with substantially less equipment than past designs, but we find that while Long Hop networks do have high performance, they are no better than random graphs, and sometimes worse.

**Slim Fly** [4] It was noted in [4] that Slim Fly's key advantage is path length. We observe in Figure 9 that the topology indeed has extremely short paths – about $85$-$90\%$ relative to the random graph – but this does not translate to higher throughput. Slim Fly's performance is essentially identical to random graphs under all-to-all and random matching TMs with a relative performance of $101\%$ using both. Relative throughput under the longest matching TM decreases with scale, dropping to $80\%$ at the largest size tested. Hence, throughput performance cannot be predicted solely based on the average path length.

#### *2) Non-Uniform Weight Synthetic Workloads*

In order to understand the robustness of throughput performance against variations in TM, we consider the response of topologies to non-uniform traffic matrices. Using the longest matching TM, we assign a weight of $10$ to $x\%$ of flows in the network which are chosen randomly while all other flows have a weight of $1$. Thus, we have a longest matching TM modified so that a fraction of flows have higher demand than the rest. We vary $x$ to understand the response of topologies to skewed traffic matrices. The experiment is conducted on a representative sample from each topology class. Note that the relative throughput at $0\%$ (which is not plotted) will be equal to that at $100\%$ since all flows are scaled by the same factor.

We observe in Figures 10 and 11 that all topologies except fat trees perform well when there are a few large flows in the network. To further understand the anomaly with fat trees, we look at the absolute throughput in fat tree, hypercube and Jellyfish network built with matching gear in Figure 12. Trends of other networks are similar to hypercube. The anomalous behavior of fat trees arises due to the fact that load on links connected to the top-of-rack switch (ToR) in fat trees is from the directly connected servers only. In all other networks, under longest matching, links connected to ToRs carry traffic belonging to flows that do not originate or terminate at the ToR. Since the load is spread more evenly, a few large flows do not affect the overall throughput. Note that fat trees can be built to be non-blocking, with high performance for any traffic; what these results show is that for these TMs, fat trees would pay a high price in equipment cost to achieve full non-blocking throughput, relative to other topology options.

**Overall,** we find that under traffic matrices with non-uniform demands, the Jellyfish, DCell, Long hop and Slim Fly topologies perform better than other topologies. While all other topologies exhibit graceful degradation of throughput with increase in the percentage of large flows, fat trees give anomalous behavior due to the load on the ToRs.

### *B. Real-world workloads*

Roy et al. [34] presents relative traffic demands measured during a period of $24$ hours in two $64$-rack clusters operated by Facebook. The first TM corresponds to a Hadoop cluster and has nearly equal weights. The second TM from a frontend cluster is more non-uniform, with relatively heavy traffic at the cache servers and lighter traffic at the web servers. Since the raw data is not publicly available, we processed the paper's plot images to retrieve the approximate weights for inter-rack traffic demand with an accuracy of $10^i$ in the interval $[10^i, 10^{i+1})$ (from data presented in color-coded log scale). Since our throughput computation rescales the TM anyway, relative weights are sufficient. Given the scale factor, the real throughput can be obtained by multiplying the computed throughput with the scale factor.

We test our slate of topologies with the Hadoop cluster TM (TM-H) and the frontend cluster TM (TM-F) and the



Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.

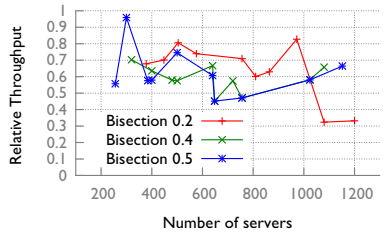



Fig. 7. HyperX Relative Throughput under Longest Matching with different Bisection

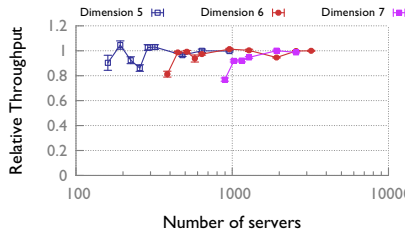



Fig. 8. Long Hop network relative throughput under longest matching TM

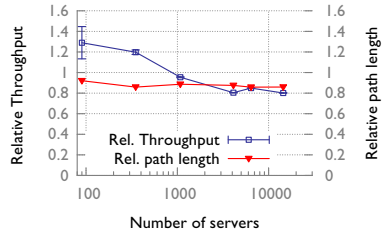



Fig. 9. Slim Fly Relative Throughput under Longest Matching TM

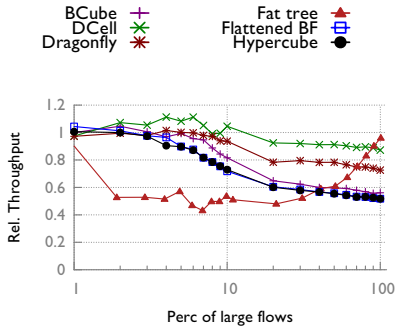



Fig. 10. Comparison of tunable TMs on topologies

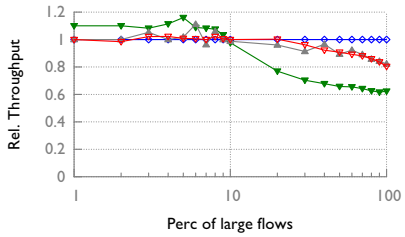



Fig. 11. Comparison of tunable TMs on topologies

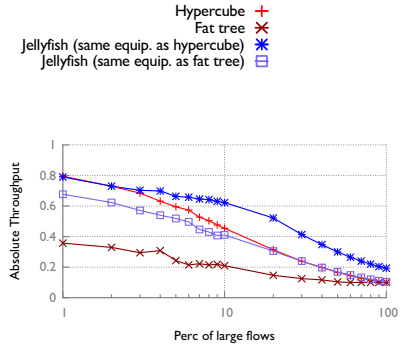



Fig. 12. Comparison of absolute throughput under tunable TMs (Jellyfish network corresponding to same equipment as hypercube and fat tree used for comparison)

results are presented in Figures 13 and 14 respectively. When a topology family does not have a candidate with 64 ToRs, the TM is downsampled to the nearest valid size (denoted as Sampled points in the plot).

We also consider modifying these TMs by shuffling the order of ToRs in the TM (denoted as Shuffled points in the plot). Under the nearly uniform TM-H, shuffling does not affect throughput performance significantly (Figure 13). However, under TM-F with skewed weights, shuffling can give significant improvement in performance. In all networks except Jellyfish, Long Hop, Slim Fly and fat trees, randomizing the flows, which in turn spreads the large flows across the network, can improve the performance significantly when the traffic matrix is non-uniform. Note that relative performance here may not match that in Figures 5 and 6 since the networks have different numbers of servers and even ToRs (due to downsampling required for several topologies). However, we observe that the relative performance between topologies remains consistent across both the traffic matrices.

### C. Summary

We have experimentally shown that:

- Cuts (as estimated using the best of a variety of heuristics) differ significantly from worst-case throughput in the majority of networks.
- Our longest matching TM approaches the theoretical lower bound in all topology families. They are significantly more difficult to route than all-to-all and random matching TMs.
- At large sizes, Jellyfish, Long Hop and Slim Fly networks provide the best throughput performance.
- Designing high throughput networks based on proxies for throughput may not yield the expected results (demonstrated with bisection bandwidth in HyperX).
- All networks except fat trees have graceful degradation in throughput under non-uniform TMs when the fraction of large flows increases. Poor performance of fat trees is due to heavy load at the ToRs.
- Randomizing the placement of heavy flows could improve the performance for throughput-intensive applications.

## V. Comparison with Related Work

The literature on network topology design is large and growing, with a number of designs having been proposed in the past few years [3], [8], [9], [11], [14]–[16], [30], [38], [39], [44]. However, each of these research proposals only makes a comparison with one or two other past proposals, with no standard benchmarks for the comparison. There has been no rigorous evaluation of a large number of topologies.

Popa et. al. [32] assess 4 topologies to determine the one that incurs least expense while achieving a target level of performance under a specific workload (all-to-all traffic). Their attempts to equalize performance required careful calibration, and approximations still had to be made. Accounting for the different costs of building different topologies is an inexact process. We sidestep that issue by using the random graph as a normalizer: instead of attempting to match performance,



Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.

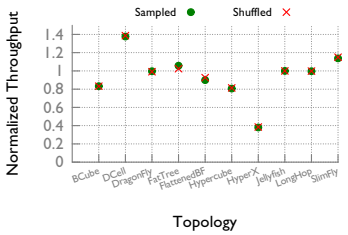



Fig. 13. Comparison of topologies with TM-H( [34])

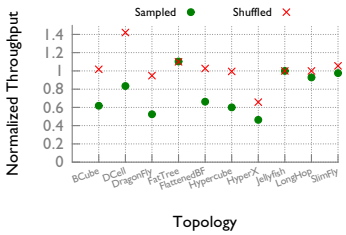



Fig. 14. Comparison of topologies with TM-F( [34])

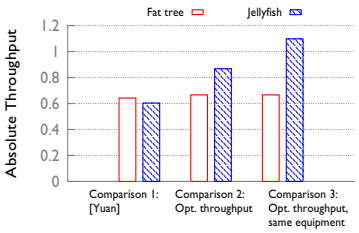



Fig. 15. Throughput comparison of Fat Tree and Jellyfish based on [47], showing the effect of two methodological changes

for each topology, we build a random graph with *identical equipment*, and then compare throughput performance of the topology with that of the random graph. This alleviates the need for equalizing configurations, thereby making it easy for others to use our tools, and to test arbitrary workloads.

Yuan et al. [47] compared the performance of fat trees and Jellyfish with HPC traffic and a specific routing scheme, concluding they have similar performance, which disagrees with our results. We replicated the method of [47] and were able to reproduce their result of similar throughput using the A2A TM (Fig. 15, Comparison 1). Their method splits flows into sub-flows and routes them along certain paths according to a routing scheme called LLSKR. It then estimates each subflow's throughput by counting and inverting the maximum number of intersecting subflows at a link along the path. Our LP-based exact throughput computation with the same LLSKR path restrictions yields slightly higher throughput in fat trees and substantially higher in Jellyfish, now $30\%$ greater than fat trees (Fig. 15, Comparison 2). This is even though we now maximize the *minimum* flow, while [47] measured the average. A second issue with the comparison in [47] is that more servers are added to Jellyfish ($160$ servers) than to fat tree ($128$ servers) with $80$ switches in each topology. Equalizing all equipment, with $80$ switches and $128$ servers in both topologies, increases the performance gap to $65\%$ (Comparison 3).

In a later work, Yuan et. al. [46] compared performance of multiple topologies. The throughput calculation was similar to [47] discussed above so it may differ from an exact LP-based method. In addition, [46] uses single-path routing. As pointed out in [46], single-path routing can perform significantly differently than multipath. Moreover, in modern enterprise private cloud data centers as well as hyperscale public clouds, multipath routing (most commonly ECMP) is standard best practice [14], [26], [36] in order to achieve acceptable performance. In contrast to [46], our study measures performance with the best possible (multipath) flow routing, so that our results reveal the topology's limits rather than a routing scheme's limits. Our LP-based method could easily be extended to incorporate path restrictions if these were desired (as we did in the previous paragraph). In addition, we study worst-case and real-world measured TMs.

Other work on comparing topologies is more focused on reliability and cuts in the topology [27]. For instance, REWIRE [8] optimizes its topology designs for high sparsest cut, although it refers to the standard sparsest cut metric as bisection bandwidth. Tomic [41] builds topologies with the objective of maximizing bisection bandwidth (in a particular class of graphs). Webb et. al [45] use bisection bandwidth to pick virtual topologies over the physical topology. An interesting point of note is that they consider all-to-all traffic "a worst-case communication scenario", while our results (Figure 4) show that other traffic patterns can be significantly worse. PAST [40] tests 3 data center network proposals with the same sparsest cut (while referring to it as bisection bandwidth). PAST finds that the throughput performance of topologies with the same sparsest cut is different in packet-level simulations, raising questions about the usefulness of such a comparison; one must either build topologies of the same cost and compare them on throughput (as we do), or build topologies with the same performance and compare cost (as Popa et. al [32] do). These observations underscore the community's lack of clarity on the relationship between bisection bandwidth, sparsest cut, and throughput. A significant component of our work tackles this subject.

## VI. Conclusion

Although high throughput is a core goal of network design, measuring throughput is a subtle problem. We have put forward an improved benchmark for throughput, including a near-worst-case traffic matrix, and have used this metric to compare a large number of topologies. When designing networks for environments where traffic patterns may be unknown or variable and performance requirements are stringent, we believe evaluation under the near-worst-case longest matching TM will be useful. We also evaluate real-world workloads and show that randomizing the traffic matrix can lead to significant improvement in performance of non-uniform workloads.

Our findings also raise interesting directions for future work. First, our longest-matching traffic heuristically produces near-worst case performance, but does not always match the lower bound. Is there an efficient method to produce even-worse-case traffic for any given topology, or provably approximate it within less than $2\times$? Second, can we leverage the result that rack-level randomization of workload placement can improve performance to provide better task placement in data centers, taking into account other practical restrictions on task placement?



Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.

## REFERENCES


[1] Topology evaluation tool. www.github.com/ankitsingla/topobench.

[2] J. H. Ahn, N. Binkert, A. Davis, M. McLaren, and R. S. Schreiber. Hyperx: Topology, routing, and packaging of efficient large-scale networks. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis*, SC '09, 2009.

[3] M. Al-Fares, A. Loukissas, and A. Vahdat. A scalable, commodity data center network architecture. In *SIGCOMM*, 2008.

[4] M. Besta and T. Hoefler. Slim Fly: A Cost Effective Low-Diameter Network Topology. In *IEEE/ACM International Conference on High Performance Computing, Networking, Storage and Analysis (SC14)*, Nov. 2014.

[5] L. N. Bhuyan and D. P. Agrawal. Generalized hypercube and hyperbus structures for a computer network. *IEEE Tran. on Computers*, 1984.

[6] S. Chawla, R. Krauthgamer, R. Kumar, Y. Rabani, and D. Sivakumar. On the hardness of approximating multicut and sparsest-cut. *computational complexity*, 2006.

[7] C. Chekuri. Routing and network design with robustness to changing or uncertain traffic demands. *SIGACT News*, 38(3):106–129, Sept. 2007.

[8] A. Curtis, T. Carpenter, M. Elsheikh, A. Lopez-Ortiz, and S. Keshav. REWIRE: An optimization-based framework for unstructured data center network design. In *INFOCOM, 2012 Proceedings IEEE*, pages 1116–1124, March 2012.

[9] A. R. Curtis, S. Keshav, and A. Lopez-Ortiz. LEGUP: using heterogeneity to reduce the cost of data center network upgrades. In *CoNEXT*, 2010.

[10] P. Elias, A. Feinstein, and C. Shannon. A note on the maximum flow through a network. *Information Theory, IEEE Transactions on*, 2(4):117–119, Dec 1956.

[11] N. Farrington, G. Porter, S. Radhakrishnan, H. H. Bazzaz, V. Subramanya, Y. Fainman, G. Papen, and A. Vahdat. Helios: A hybrid electrical/optical switch architecture for modular data centers. In *SIGCOMM*, 2010.

[12] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399 –404, 1956.

[13] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. 1979.

[14] A. Greenberg, J. R. Hamilton, N. Jain, S. Kandula, C. Kim, P. Lahiri, D. A. Maltz, P. Patel, and S. Sengupta. VL2: A Scalable and Flexible Data Center Network. In *SIGCOMM*, 2009.

[15] C. Guo, G. Lu, D. Li, H. Wu, X. Zhang, Y. Shi, C. Tian, Y. Zhang, and S. Lu. BCube: a high performance, server-centric network architecture for modular data centers. *SIGCOMM Comput. Commun. Rev.*, 39(4):63–74, Aug. 2009.

[16] C. Guo, H. Wu, K. Tan, L. Shi, Y. Zhang, and S. Lu. DCell: A scalable and fault-tolerant network structure for data centers. In *SIGCOMM*, 2008.

[17] Gurobi Optimization Inc. Gurobi optimizer reference manual. http://www.gurobi.com, 2013.

[18] T. Hoefler, T. Schneider, and A. Lumsdaine. The impact of network noise at large-scale communication performance. In *Parallel Distributed Processing, 2009. IPDPS 2009. IEEE International Symposium on*, pages 1–8, May 2009.

[19] A. Jokanovic, G. Rodriguez, J. C. Sancho, and J. Labarta. Impact of inter-application contention in current and future hpc systems. In *High Performance Interconnects (HOTI), 2010 IEEE 18th Annual Symposium on*, pages 15–24, Aug 2010.

[20] A. Jokanovic, J. C. Sancho, J. Labarta, G. Rodriguez, and C. Minkenberg. Effective quality-of-service policy for capacity high-performance computing systems. In *High Performance Computing and Communication 2012 IEEE 9th International Conference on Embedded Software and Systems (HPCC-ICESS), 2012 IEEE 14th International Conference on*, pages 598–607, June 2012.

[21] A. Jokanovic, J. C. Sancho, G. Rodriguez, A. Lucero, C. Minkenberg, and J. Labarta. Quiet neighborhoods: Key to protect job performance predictability. In *Parallel and Distributed Processing Symposium (IPDPS), 2015 IEEE International*, pages 449–459, May 2015.

[22] S. A. Jyothi, A. Singla, B. Godfrey, and A. Kolla. Measuring and understanding throughput of network topologies. http://arxiv.org/abs/1402.2531, 2014.

[23] J. Kim, W. J. Dally, and D. Abts. Flattened butterfly: a cost-efficient topology for high-radix networks. *SIGARCH Comput. Archit. News*, 35(2):126–137, June 2007.

[24] J. Kim, W. J. Dally, S. Scott, and D. Abts. Technology-Driven, Highly-Scalable Dragonfly Topology. In *Proceedings of the 35th Annual International Symposium on Computer Architecture*, ISCA '08, pages 77–88, 2008.

[25] M. Kodialam, T. V. Lakshman, and S. Sengupta. Traffic-oblivious routing in the hose model. *IEEE/ACM Trans. Netw.*, 19(3):774–787, 2011.

[26] P. Lapukhov, A. Premji, and J. Mitchell. Use of bgp for routing in large-scale data centers. Internet-Draft draft-ietf-rtgwg-bgp-routing-large-dc-09, IETF Secretariat, March 2016. http://www.ietf.org/internet-drafts/draft-ietf-rtgwg-bgp-routing-large-dc-09.txt.

[27] D. S. Lee and J. L. Kalb. Network topology analysis. Technical report, 2008.

[28] T. Leighton and S. Rao. Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms. *J. ACM*, 46(6):787–832, Nov. 1999.

[29] C. E. Leiserson. Fat-trees: universal networks for hardware-efficient supercomputing. *IEEE Trans. Comput.*, 34(10):892–901, Oct. 1985.

[30] R. N. Mysore, A. Pamboris, N. Farrington, N. Huang, P. Miri, S. Radhakrishnan, V. Subramanya, and A. Vahdat. Portland: A scalable fault-tolerant layer 2 data center network fabric. In *SIGCOMM*, 2009.

[31] D. Padua. *Encyclopedia of Parallel Computing*. Number v. 4 in Springer reference. Springer, 2011. See bisection bandwidth discussion on p. 974.

[32] L. Popa, S. Ratnasamy, G. Iannaccone, A. Krishnamurthy, and I. Stoica. A cost comparison of datacenter network architectures. In *CoNEXT*, 2010.

[33] B. Prisacari, G. Rodriguez, P. Heidelberger, D. Chen, C. Minkenberg, and T. Hoefler. Efficient Task Placement and Routing in Dragonfly Networks . In *Proceedings of the 23rd ACM International Symposium on High-Performance Parallel and Distributed Computing (HPDC'14)*. ACM, Jun. 2014.

[34] A. Roy, H. Zeng, J. Bagga, G. Porter, and A. C. Snoeren. Inside the social network's (datacenter) network. *SIGCOMM Comput. Commun. Rev.*, 45(5):123–137, 2015.

[35] F. Shahrokhi and D. Matula. The maximum concurrent flow problem. *Journal of the ACM*, 37(2):318–334, 1990.

[36] A. Singh, J. Ong, A. Agarwal, G. Anderson, A. Armistead, R. Bannon, S. Boving, G. Desai, B. Felderman, P. Germano, A. Kanagala, J. Provost, J. Simmons, E. Tanda, J. Wanderer, U. Hölzle, S. Stuart, and A. Vahdat. Jupiter rising: A decade of clos topologies and centralized control in google's datacenter network. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, SIGCOMM '15, pages 183–197. ACM, 2015.

[37] A. Singla, P. B. Godfrey, and A. Kolla. High throughput data center topology design. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, April 2014.

[38] A. Singla, C.-Y. Hong, L. Popa, and P. B. Godfrey. Jellyfish: Networking data centers randomly. In *NSDI*, 2012.

[39] A. Singla, A. Singh, K. Ramachandran, L. Xu, and Y. Zhang. Proteus: a topology malleable data center network. In *HotNets*, 2010.

[40] B. Stephens, A. Cox, W. Felter, C. Dixon, and J. Carter. PAST: Scalable Ethernet for data centers. In *CoNEXT*, 2012.

[41] R. V. Tomic. Optimal networks from error correcting codes. In *ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, 2013.

[42] B. Towles and W. J. Dally. Worst-case traffic for oblivious routing functions. In *Proceedings of the Fourteenth Annual ACM Symposium on Parallel Algorithms and Architectures*, SPAA '02, pages 1–8, New York, NY, USA, 2002. ACM.

[43] A. Valadarsky, M. Dinitz, and M. Schapira. Xpander: Unveiling the secrets of high-performance datacenters. In *Proceedings of the 14th ACM Workshop on Hot Topics in Networks*, HotNets-XIV, pages 16:1–16:7, New York, NY, USA, 2015. ACM.

[44] G. Wang, D. G. Andersen, M. Kaminsky, K. Papagiannaki, T. S. E. Ng, M. Kozuch, and M. Ryan. c-Through: Part-time Optics in Data Centers. In *SIGCOMM*, 2010.

[45] K. C. Webb, A. C. Snoeren, and K. Yocum. Topology switching for data center networks. In *HotICE*, 2011.

[46] X. Yuan, S. Mahapatra, M. Lang, and S. Pakin. LFTI: A new performance metric for assessing interconnect designs for extreme-scale HPC systems. In *2014 IEEE 28th International Parallel and Distributed Processing Symposium, Phoenix, AZ, USA, May 19-23, 2014*, pages 273–282, 2014.

[47] X. Yuan, S. Mahapatra, W. Nienaber, S. Pakin, and M. Lang. A New Routing Scheme for Jellyfish and Its Performance with HPC Workloads. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, SC '13, pages 36:1–36:11, 2013.




Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.

## VII. Artifact Description: Measuring and Understanding Throughput of Network Topologies

### A. Abstract

This artifact contains the code to generate the linear program (LP) for measuring throughput and the topologies used for evaluation. It also includes scripts to run the LP with the Gurobi LP solver and generate plots using gnuplot.

### B. Description

The LP generator takes as input a topology and a traffic type. In addition to the tested topologies, users can also specify their custom topologies which can be read from a file. Traffic types supported are all-to-all, random permutation and longest matching.

*1) Check-list (artifact meta information):*

- **Program: Java**
- **Compilation:javac 1.7 or higher**
- **Data set: Publicly available topologies included in the artifact**
- **Run-time environment: Linux with LP solver (Gurobi/C-PLEX) and gnuplot 4.4 or higher**
- **Hardware: Any hardware**
- **Experiment customization: Modify scripts to choose specific topology and traffic type**
- **Publicly available?: Yes**

*2) How delivered:* Topobench is an open-source test bench released under the MIT license. The test bench and related instructions are hosted on Github. (https://github.com/ankitsingla/topobench)

*3) Hardware dependencies:* The test bench will run on any general purpose computer. However, the largest network that can be tested depends on available RAM and the topology being tested. This is due to the memory restrictions of the LP solvers.

*4) Software dependencies:* Experiments in this paper were run on Ubuntu 14.04 with javac 1.8.0 and Gurobi 5.6. The scripts use gnuplot 4.4 or higher (for the pdfcairo terminal). But if you don't have it, you can still get the result files and plot them using other software.

*5) Datasets:* HPC and data center topologies evaluated in this paper are available in the folder *lpmaker/graphs* and real-world graphs are available in the folder *graphrepo*.

### C. Installation

The test bench and instructions to use it can be downloaded from the Github repository - https://github.com/ankitsingla/topobench.

### D. Experiment workflow

The experiments are run with shell scripts and the results are stored in text files.

Clone the test bench to local machine and compile:

```
$ git clone https://github.com/
    ankitsingla/topobench
$ cd topobench
$ javac lpmaker/ProduceLP.java
```

Run the scripts located in the folder *scripts*:

```
$ bash scripts/fatCompare.sh
```

Plot using gnuplot scripts:

```
$ gnuplot gnuplotscripts/fatCompare.plt
```

Clean folders:

```
$ bash scripts/clean.sh
```

### E. Evaluation and expected result

After running an experiment, the results are available in the folder *resultfiles*. The results can be plotted using gnuplot (4.4+) using scripts in the folder *gnuplotscripts*. The pdf plots are stored in the folder *plots*.

### F. Experiment customization

Scripts can be modified to choose a specific topology and traffic matrix. The experiments were run with Gurobi 5.6 and gnuplot 4.6. However, any LP solver which can read the standard LP format (e.g., CPLEX) and any plotting software may be used.



Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:30:19 UTC from IEEE Xplore. Restrictions apply.